

Breach

A Tactical Spaceship Boarding Game

Document 00: Architecture Overview & Design Decisions

Erik Idir

March 2026

Abstract

This document summarizes the core architectural decisions for **Breach**, a tactical squad-based spaceship boarding game. It serves as the foundation reference for all subsequent system-specific design documents and as a discussion basis for the development team.

Contents

1	Game Overview	3
2	Engine & Tooling	3
2.1	Engine Candidates	3
2.2	Recommendation	3
3	Two-Layer Architecture	3
4	Spatial Representation	4
4.1	Grid-Based World	4
4.2	Fine Grid with Coarse Logical Structure	4
5	State Representation	4
5.1	Tile Objects + Cached Arrays	4
5.2	Data-Driven Material System	5
5.3	State Layers	5
5.4	Why This Approach?	5
5.5	Entity List	6
6	Simulation Tick & Double Buffering	6
6.1	Turn Resolution	6
6.2	Double-Buffered Propagation	6
7	Design Document Roadmap	6
7.1	Environmental Simulation	6
7.2	Game Mechanics	7
7.3	Presentation	7
7.4	Content Design	7

8	Language Pipeline	8
8.1	Prototyping Phase: Python	8
8.2	Production Phase: C++ with Python Bindings	8
8.3	Architecture	8
8.4	Zero-Copy Array Sharing	8
8.5	Build Targets	9
8.6	Precedent	9
9	Neural Network Considerations	9
10	Open Questions & Next Steps	10

1 Game Overview

A top-down, turn-based tactical game in which squads board spaceships to complete objectives (assassination, theft, hostage rescue, etc.). The core design philosophy is **emergent complexity from simple rules**: a small number of interconnected systems—destructible environments, pressure, fire, liquids, creatures—combine to produce a vast space of possible situations.

The game is set in a satirical 2087 world ruled by central banking dynasties. Tone is darkly humorous and absurdist.

2 Engine & Tooling

The core architecture is a **headless simulation layer** fully separated from the presentation layer (Section 3). This means the engine choice affects *only* the presentation layer—rendering, input, audio, UI—while the simulation remains engine-agnostic.

2.1 Engine Candidates

Unreal Engine

Idir’s primary expertise. Powerful rendering, mature tooling, C++ native. However, Unreal is fundamentally 3D-first—heavy for a 2D tile-based game with significant overhead that goes unused. Makes more sense if the project later moves to 2.5D or 3D.

Unity

Excellent 2D support with a built-in 2D lighting system that supports normal maps natively. Large asset store and community for 2D game development. Lighter weight than Unreal for 2D projects. The simulation is C++ regardless; Unity calls it via native plugin. Licensing terms should be reviewed.

Godot

Free, open source, lightweight, with excellent 2D support. Built-in `CanvasItem` shaders with normal map support. C++ integration via `GDEXTENSION`. Smaller ecosystem than Unity/Unreal but very well suited to 2D games with dynamic lighting.

Custom Engine

Total control, no licensing issues, educational value. But an enormous time investment on solved problems (rendering, input, audio, UI, asset loading). Not recommended unless engine development is a goal in itself.

2.2 Recommendation

For a 2D tile-based game with dynamic lighting and normal-mapped sprites, **Unity or Godot** are the strongest candidates. Both have native 2D lighting pipelines that match our design requirements (see Document 13: Graphics, Lighting & Art Pipeline). Unreal remains viable—especially given Idir’s expertise—but the 2D lighting workflow may be smoother in Unity or Godot.

Decision status: Open—to be discussed and confirmed with Idir before prototyping begins.

3 Two-Layer Architecture

Simulation Layer

A self-contained simulation that owns all game rules and state. This layer has **no dependency on any game engine** and can run headless for AI training and automated testing. It exposes a clean interface: `get_state()`, `apply_action()`, `step()`.

Presentation Layer

The game engine reads simulation state each frame and renders it. Player input is captured by the engine, translated into game actions, and forwarded to the simulation.

Neural Network Training

The headless simulation generates training data and runs self-play at scale without spinning up a rendering engine.

This separation allows parallel development: simulation logic can progress independently of visual implementation.

4 Spatial Representation

4.1 Grid-Based World

The game world is represented as a **uniform 2D grid**. Every structural element (walls, floors, hull, doors, windows) occupies one or more grid cells. This was chosen over continuous geometry for the following reasons:

- All propagation systems (pressure, fire, liquid) become cellular automata—simple, fast, and predictable.
- Game state maps directly to a 2D tensor for neural network input (stacked feature planes, cf. AlphaStar).
- Pathfinding, line-of-sight, and cover calculations are straightforward on a grid.
- Serialization, replay, and debugging are trivial.

4.2 Fine Grid with Coarse Logical Structure

A single player entity occupies a **3×3 block** of tiles. The grid resolution is therefore finer than the logical “square” a character stands in. This gives us:

- Smoother movement and sub-positioning (center, edges, corners).
- Granular destruction—a wall segment can have a small hole, a large hole, or be fully destroyed.
- Natural partial cover and line-of-sight without a separate high-resolution system.
- Better visual fidelity for gore, splatter patterns, and debris.

Room layout, doors, and corridors are still designed on the **coarse grid** (each coarse cell = 3×3 fine tiles). The fine grid provides resolution *within* that structure.

Implementation note: Grid resolution should be a configurable parameter. Initial development can use 1:1 (one tile per logical cell) and scale to 3:1 once core systems are stable.

5 State Representation

5.1 Tile Objects + Cached Arrays

The map is a 2D grid of **Tile objects**—each tile stores its material, hit points, fire state, liquid contents, and a list of entities. This object-based representation is easy to extend (adding

a property means adding a field, not changing every tensor consumer) and naturally handles mixed state (type information alongside continuous values like fire intensity).

For physics and propagation, the simulation maintains **cached arrays** derived from tile state: light-blocking, gas-blocking, walkability, flammability. These caches are rebuilt on tile change (`on_tile_changed(x, y)`) and allow all propagation systems to operate as fast bulk array operations.

This hybrid gives us the best of both worlds: rich per-tile data for game logic, and fast array operations for physics.

5.2 Data-Driven Material System

Tile behavior is determined by **material type**. All material properties are defined in a single lookup table—adding a new material requires one table entry and zero code changes:

Material	Max HP	Flammable	Blocks Light	Blocks Gas	Blast Resist.
None	0	No	No	No	0.0
Steel	200	No	Yes	Yes	0.8
Wood	60	Yes	Yes	Yes	0.2
Glass	15	No	No	Yes	0.0
Hull	300	No	Yes	Yes	0.9

Table 1: Material properties. Each tile’s behavior derives entirely from its material. The level editor paints materials; all properties auto-derive.

5.3 State Layers

The following fields are tracked per tile, either as Tile object properties or as cached arrays for bulk computation:

Field	Storage	Description
Material	Tile object	Determines all structural properties
Hit points	Tile object	Per-tile structural HP
Pressure	Cached array	Atmospheric pressure (gradual decompression)
Fire intensity	Tile object	Heat / flame level
Liquid type	Tile object	None, blood, water, fuel ...
Liquid depth	Tile object	Volume of liquid in tile
Smoke density	Cached array	Obscures vision, spreads through gaps
Oxygen	Cached array	Supports suffocation, consumed by fire
Temperature	Tile object	Thermal state
Entity list	Tile object	Creatures/players occupying tile

Table 2: Per-tile state fields. “Cached array” fields are also stored per-tile but maintained as contiguous arrays for physics computation. Additional layers may be added per system document.

5.4 Why This Approach?

- **Game logic as array operations.** Explosions, pressure propagation, and fire spread operate on cached arrays—fast bulk computation via NumPy (prototype) or Eigen/raw arrays (C++).

- **Emergent interactions are automatic.** Each system reads shared arrays and writes to its own. Fire checks the liquid array for fuel; creatures check it for blood. No explicit cross-system wiring.
- **ML-ready.** Stack the cached arrays into channels and feed directly to a CNN—no conversion step.
- **Extensible.** Adding a new tile property is a field addition, not a tensor schema change.
- **Serializable.** Dump tile objects $\rightarrow$ complete snapshot. Save/load, replay, and undo come nearly for free.

5.5 Entity List

Mobile entities (squad members, creatures, items) are stored in a **separate entity list**, since they carry properties that do not map well to spatial grids: inventory, action points, status effects, allegiance, AI state. Their position is an (x, y) index into the grid.

6 Simulation Tick & Double Buffering

6.1 Turn Resolution

Each player turn proceeds as follows:

1. Player (or AI) selects and submits actions.
2. Actions are applied as direct state modifications (damage, movement, item use, etc.).
3. The simulation runs N propagation ticks to let the environment settle (pressure equalization, fire spread, liquid flow, creature reactions).
4. The new steady-state is presented; the next player's turn begins.

6.2 Double-Buffered Propagation

All propagation systems resolve **simultaneously** within each tick. This is implemented via double buffering:

1. Each cached array has a *current* buffer (read-only during the tick) and a *next* buffer (write target).
2. Every system reads from *current* and writes to *next*.
3. At the end of the tick, *next* becomes *current* (pointer swap).

This avoids order-of-operations bugs and produces physically more accurate behavior.

7 Design Document Roadmap

Each system is specified in its own design document. Documents progress through: **brainstorm** (Markdown notes) $\rightarrow$ **design document** (L^AT_EX) $\rightarrow$ **implementation**.

7.1 Environmental Simulation

Matrix-based physics and propagation systems—the cellular automata that drive emergent gameplay.

Doc	System	Primary State (read/write)	Status
01	Structure, Materials & Destruction	Material (rw), HP (rw), Pressure (w)	Designed
02	Pressure & Decompression	Pressure (rw), Oxygen (rw), Gas-block (r)	Designed
03	Fire Propagation	Fire (rw), Oxygen (r), Liquid (r), HP (w)	Brainstorm
04	Liquids (blood, water, fuel)	Liquid type/depth (rw), Pressure (r)	Brainstorm
05	Smoke Propagation	Smoke (rw), Pressure (r), Gas-block (r)	Designed
06	Explosions	Pressure (w), HP (w), Fire (w), Smoke (w)	Designed
07	Line of Sight & Cover	Light-block (r), Fire (r), Smoke (r)	Brainstorm

Table 3: Environmental simulation documents. “Designed” indicates that core equations and algorithms have been worked out in brainstorm notes; the L^AT_EX design document is pending.

7.2 Game Mechanics

Higher-level rules and systems that run on top of the environmental simulation.

Doc	System	Scope	Status
08	Turn System & Simultaneous Turns	Turn structure, simultaneous resolution, action points	Brainstorm
09	Combat Mechanics	Damage, weapons, hit calculation	Brainstorm
10	Economy & Currency	CBD / Gold / Bitcoin, inflation, tracking	Brainstorm
11	Inventory & Equipment	Loadout, weight, item interactions	Brainstorm
12	Player Interface & Interaction	Controls, HUD, action selection, feedback	Not started

Table 4: Game mechanics documents.

7.3 Presentation

Doc	System	Scope	Status
13	Graphics, Lighting & Art Pipeline	2D raycasting, normal maps, shadow stealth, art phases	Designed

Table 5: Presentation layer documents.

7.4 Content Design

Game content that populates the simulation—enemies, missions, levels, and narrative. These documents are relatively independent of the simulation architecture.

Doc	System	Scope	Status
14	Creatures & AI Behaviors	Swarms, predators, genetic sol- diers, AI design	Brainstorm
15	Missions & Objectives	Mission types, objectives, extrac- tion/sabotage	Brainstorm
16	Ship Layouts & Level Design	Map design, room types, level ed- itor	Brainstorm
17	Narrative & Lore	Satirical 2087 world, Water- gate/Princes plot, factions	Designed

Table 6: Content design documents.

8 Language Pipeline

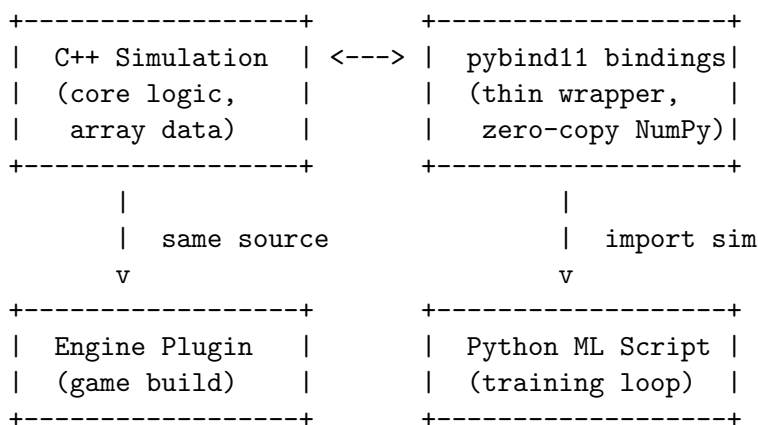
8.1 Prototyping Phase: Python

The simulation is prototyped in **Python with NumPy**. This enables rapid iteration on game rules, physics parameters, and system interactions before committing to a compiled language. All propagation systems (explosions, decompression, smoke) have been designed and validated as NumPy array operations.

8.2 Production Phase: C++ with Python Bindings

Once core systems are validated, the simulation moves to **C++** for performance (pathfinding, propagation, and bulk array operations at thousands of steps per second). The ML training pipeline remains in **Python** (PyTorch). The two are connected via **pybind11** (or nanobind), exposing the C++ simulation as an importable Python module with zero-copy array sharing.

8.3 Architecture



8.4 Zero-Copy Array Sharing

The C++ simulation class owns the raw array data. `pybind11` wraps these buffers as NumPy arrays *without copying*—both languages point at the same memory. From Python, accessing `env.state.pressure` returns a standard NumPy array that can be fed directly into a neural network.

A typical training loop:

```
import sim          # compiled C++ module
```



```

env = sim.BoardingEnv(map="frigate_01")

obs = env.reset() # returns dict of NumPy arrays (zero-copy)
for step in range(max_steps):
    action = agent.act(obs)
    obs, reward, done, info = env.step(action)
    agent.learn(obs, reward, done)

```

This follows the standard Gymnasium interface, making it compatible with existing RL libraries (Stable Baselines3, RLlib, CleanRL, etc.).

8.5 Build Targets

The C++ simulation compiles into two targets from the **same source code**:

Engine Plugin

Compiled as a module within the game engine project. The presentation layer calls directly into the simulation via C++ APIs.

Python Module (.so / .pyd)

Built with CMake + pybind11. Produces an importable Python package for headless training, evaluation, and analysis.

This ensures that the game and the training environment run *identical* simulation logic—no translation bugs, no drift between the two.

8.6 Precedent

This pattern is standard in high-performance RL research. Notable examples include DeepMind’s MuJoCo bindings, OpenAI Gym’s Atari environments, and PettingZoo’s multi-agent environments—all C++ simulations with thin Python wrappers.

9 Neural Network Considerations

The array-based state representation is designed to be directly consumable by convolutional neural networks, following the stacked feature-plane approach used by AlphaStar (StarCraft II).

- Each cached array becomes one (or more) input channels.
- For training, the fine grid can be downsampled to coarse resolution if full resolution proves unnecessary.
- The headless simulation enables massively parallel self-play without rendering overhead.
- Action space definition and reward shaping are deferred to a future document.

Design principle: No architectural decision should foreclose the option of training a neural network agent. The grid representation, clean state serialization, and headless simulation capability are all in service of this goal.

10 Open Questions & Next Steps

1. **Engine choice:** Review engine candidates with Idir and select one (Section 2).
2. **Presentation layer integration:** Once the engine is chosen—how does the presentation layer poll or subscribe to simulation state? Tile rendering approach? Sprite/animation pipeline?
3. **Convert brainstorm notes to design documents:** Prioritize Documents 01–07 (environmental simulation), then 08–12 (game mechanics).
4. **Prototype:** Build the Python/NumPy simulation prototype with core environmental systems.
5. **Establish shared repository** and document conventions.
6. **Network replication strategy** if multiplayer is pursued.